Universidad Mariano Galvez

Facultad de Ingeniería Matemática y Ciencias Físicas

Ingeniería Eléctrica e Ingeniería Mecánica

Programación para la Ciencia y la Ingeniería II

Ing. Carlos Alberto Arias López

Proyecto 2

014-25-22072	FERNANDO DAVID GONZÁLEZ TZUN
014-25-14095	OSCAR ALEXANDER GONZÁLEZ PÉREZ
1492-24-27134	DIEGO JOSUE RIVERA MONROY
014-25-16673	JUAN PABLO ERNESTO POSADAS SOBERANIS
014-25-22860	IVONNE GENESIS AIDE PEREZ AGUILAR
014-25-8251	JOSÉ ALEJANDRO CABALLEROS TORRES

Introducción

En el presente ensayo se documenta el proceso de desarrollo de un sistema de gestión académica. El sistema desarrollado permite la gestión de usuarios con tres roles distintos (administrador, tutor y estudiante), la administración de cursos, la configuración de horarios de tutoría mediante archivos XML, el ingreso y gestión de notas a través de matrices dispersas, y la generación de reportes gráficos. A lo largo del desarrollo, se aplicaron conceptos fundamentales de programación como manejo de variables, encapsulamiento mediante clases, estructuras de datos lineales y no lineales, así como patrones de comunicación entre servicios mediante API REST.

1. Gestión de Variables y Persistencia de Datos

Uno de los aspectos fundamentales en el desarrollo del sistema fue el manejo adecuado de las variables para garantizar la integridad de los datos a través de los diferentes módulos. En el contexto de una aplicación web con autenticación de usuarios y las variables de sesión.

Las sesiones en Django se implementaron mediante el diccionario **request.session**, que permite almacenar variables específicas por usuario a lo largo de su navegación. Las variables principales gestionadas fueron:

```
request.session['rol'] = 'admin'        # Tipo de usuario
request.session['nombre'] = 'Admin'     # Nombre del usuario
request.session['usuario'] = 'Admin'    # Identificador de acceso
request.session['registro'] = '201'     # Para tutores
request.session['carnet'] = '101'       # Para estudiantes
```

Estas variables demostraron ser esenciales para controlar el acceso a las diferentes secciones del sistema y personalizar la experiencia del usuario. La correcta inicialización y destrucción de estas variables en el proceso de logout garantiza la seguridad de la aplicación.

En el backend desarrollado con Flask, se utilizaron variables de entorno y configuración para parametrizar aspectos como las rutas de archivos, credenciales del administrador y URLs de conexión entre servicios. La variable **BACKEND_URL** en Django y **host='0.0.0.0**' en Flask permitieron la comunicación entre diferentes equipos en red.

2. Clases y Programación Orientada a Objetos

La programación orientada a objetos fue implementada principalmente en el módulo de matriz dispersa para el manejo de notas. La clase MatrizDispersa encapsula la lógica de almacenamiento y consulta de calificaciones.

```python
class MatrizDispersa:
    def __init__(self):
        self.datos = {}      # Diccionario anidado
        self.filas = set()   # Actividades
        self.columnas = set() # Carnets

    def agregar(self, actividad, carnet, nota):
        if 0 <= nota <= 100:
            self.datos[(actividad, carnet)] = nota
            self.filas.add(actividad)
            self.columnas.add(carnet)
            return True
        return False

    def promedio_por_actividad(self, actividad):
        notas = list(self.obtener_por_actividad(actividad).values())
        return sum(notas) / len(notas) if notas else 0
```

Esta estructura permitió almacenar notas de manera eficiente, evitando matrices densas con muchos valores nulos. La elección de un diccionario con tuplas como clave (actividad, carnet) resultó óptima para consultas rápidas por actividad o por estudiante.

En el backend de Flask, se utilizaron Blueprints para modularizar las rutas:

```python
admin_bp = Blueprint("admin", __name__)
tutor_bp = Blueprint("tutor", __name__)
auth_bp = Blueprint("auth", __name__)
```

3. Estructuras de Datos Avanzadas

3.1 Matriz Dispersa para Notas

La matriz dispersa implementada resolvió el problema de almacenar calificaciones donde no todos los estudiantes tienen todas las actividades. Con una matriz tradicional de N×M, el espacio requerido crecería proporcionalmente al producto del número de estudiantes por el número de actividades. La matriz dispersa solo almacena las combinaciones existentes, ahorrando recursos computacionales.

3.2 Almacenamiento en JSON

Las notas se serializan a JSON para persistencia:

```python
def to_dict(self):
    return {
        "datos": {f"{k[0]}|{k[1]}": v for k, v in self.datos.items()},
        "filas": list(self.filas),
        "columnas": list(self.columnas)
    }
```

Esta representación permite guardar la matriz en disco y recuperarla posteriormente.

3.3 Listas Enlazadas

Aunque no se implementaron directamente en el código final, durante las fases iniciales del proyecto se exploraron listas enlazadas para gestionar los horarios de tutoría, comprendiendo sus ventajas para inserciones dinámicas y desventajas en acceso aleatorio.

4. Comunicación entre Servicios (API REST)

Uno de los desafíos técnicos más complejos fue establecer comunicación entre el frontend (Django) y el backend (Flask). La solución adoptada fue implementar una API REST en Flask que expone endpoints para las operaciones principales:

Endpoint	Método	Función
`/api/login`	POST	Autenticación de usuarios
`/api/cursos/<registro>`	GET	Obtener cursos de un tutor
`/api/horarios/<registro>`	GET	Obtener horarios guardados
`/api/horarios/cargar/<registro>`	POST	Cargar horarios desde XML
`/api/notas/<registro>`	POST	Cargar notas desde XML
`/api/reportes/<registro>`	GET	Obtener datos para gráficos

El uso de requests en Django para consumir estos endpoints permitió mantener la separación de responsabilidades:

```
response = requests.post(
    f"{BACKEND_URL}/login",
    json={"usuario": usuario, "contrasenia": contrasenia},
    timeout=5
)
```

5. Manejo de Archivos XML

El sistema procesa dos tipos de archivos XML:

Horarios (formato):

```
<horarios>
    <horario>
        <curso>770</curso>
        <horario_inicio>08:00</horario_inicio>
        <horario_fin>10:00</horario_fin>
    </horario>
</horarios>
```

Notas (formato):

```
<curso codigo="770">
    <actividad nombre="Examen 1">
        <nota carnet="101">85</nota>
    </actividad>
</curso>
```

La validación de los archivos incluye verificación de cursos existentes, formato de horas (HH:MM) y rango de notas (0-100). Las notas inválidas se descartan con mensajes informativos.

6. Generación de Reportes Gráficos

Los reportes se implementaron utilizando Chart.js en el frontend, consumiendo datos de la API del backend:

Promedio por actividades: Gráfico de barras que muestra el promedio de calificaciones para cada actividad.

TOP de notas: Gráfico de barras ordenado de mayor a menor para una actividad seleccionada.

La funcionalidad de exportación a PDF se implementó con html2pdf, permitiendo a los tutores generar reportes descargables.

Conclusiones

El desarrollo de este sistema de gestión académica permitió aplicar y consolidar conceptos fundamentales de programación:

1. Variables y sesiones: La correcta gestión de variables de sesión es crucial para aplicaciones web con múltiples roles de usuario.
2. Clases y POO: La encapsulación mediante clases facilita el mantenimiento y la reutilización del código.
3. Estructuras de datos: La matriz dispersa demostró ser una solución eficiente para el almacenamiento de calificaciones.
4. APIs REST: La separación entre frontend y backend mediante API REST permite escalabilidad y distribución del sistema.
5. Validación de datos: La verificación de entradas (notas en rango, formato de horas, existencia de cursos) es esencial para la integridad del sistema.
6. XML como formato de intercambio: Los archivos XML facilitan la carga masiva de datos (horarios y notas) con una estructura autodescriptiva.
7. Gráficos interactivos: La visualización de datos mediante Chart.js mejora la experiencia del usuario y facilita la interpretación de la información.